Class 1: Introduction to Course — MLOps, Agile, Data Mining

Data-driven Systems Engineering

Agenda

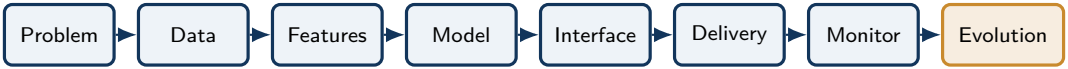
- Course positioning: what “data-driven systems engineering” means in practice.
- The three macro-blocks of the 36-session course: Python, Data-Driven, MLOps.
- Why ML projects fail when they stop at the notebook.
- The course technology stack: Python, Jupyter, pandas, scikit-learn, Flask, Streamlit, River, Docker, Jenkins, GitHub Actions.
- Agile AI and data mining: how to structure work when data, labels, and requirements all move.
- Running cases from this repository: fraud detection and pasteurization monitoring.

Why this course exists

Learning goals

- Connect data analysis, software engineering, and operations in one engineering workflow.
- Move from notebook-level experimentation to deployable and monitorable systems.
- Understand what each tool in the stack is for and when not to use it.
- Learn to discuss ML systems as socio-technical systems, not only as models.

Course Map and Running Cases



Today in the course

Focus: End-to-end lifecycle thinking for data-driven systems.
Bridge: We start from problem framing and follow the path all the way to deployment, monitoring, and evolution.

Fraud case

Fraud detection becomes a full service: data understanding, feature design, scoring API, release process, and monitoring.

Pasteurization case

Pasteurization becomes a monitoring system: synthetic sensors, forecasting or alerting logic, dashboard views, and drift-aware operation.

Course thread: This class defines the vocabulary and the two running cases used across the rest of the course.

What is data-driven systems engineering?

- A system is more than code: it includes data sources, interfaces, operators, processes, and constraints.
- Data-driven means that the system behavior depends partly on learned patterns rather than only on hand-written rules.
- Engineering means we care about reliability, maintainability, performance, and traceability.

Core message

The course is not about "how to train one model". It is about how to design a system that uses data and models responsibly over time.

Course arc: three macro-blocks, 36 sessions

1. **Python block (Classes 1–12):** programming instrumentation — language basics, data structures, functions, classes, files, pandas mechanics, first contact with scikit-learn, and EDA practice.
2. **Data-Driven block (Classes 13–20):** data types and representation, encoding, feature engineering, modeling, and online learning with concept-drift practice.
3. **MLOps block (Classes 21–36):** software engineering, requirements, model serving, process models, agile methods, delivery pipelines, monitoring, and project work.

Guiding question

How do we turn a promising model into a reliable data product that a team can build, operate, and improve?

Why notebook success is not production success

In a notebook

- Small sample runs
- Manual sequencing
- Informal assumptions
- One main user

In production

- Repeated requests
- Input validation
- Monitoring and logs
- Many users and stakeholders

Discussion prompt

If a model has 95% accuracy in a notebook but crashes on malformed inputs, is it useful to the organization?

Running cases from this repository

Fraud detection

- Tabular batch data
- Heavy class imbalance
- Binary classification
- API and dashboard deployment
- Human review after alerts

Pasteurization monitoring

- Synthetic streaming sensor data
- Real-time forecasting
- Drift detection
- Industrial process states
- Continuous visualization

Repo activity

These two cases cover most of the course: data preparation, model design, serving, CI/CD, monitoring, and discussion of human decisions.

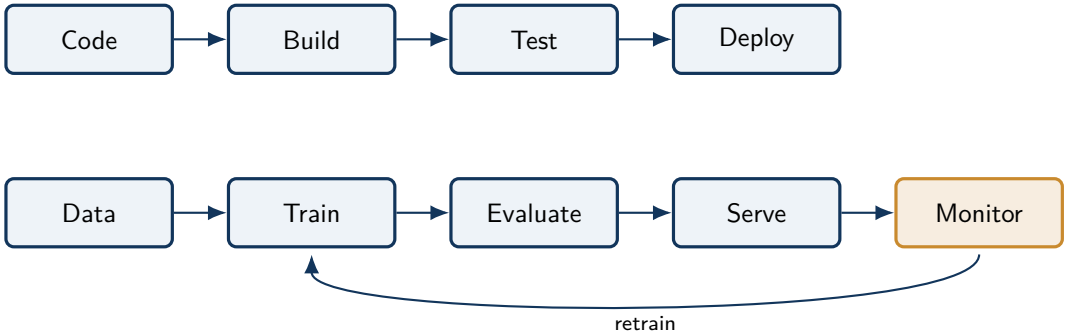
What MLOps is really about

- MLOps extends software delivery with data validation, experiment tracking, model packaging, deployment, monitoring, and retraining.
- It reduces friction between data scientists, software engineers, and operations.
- It makes model behavior observable after release.
- It creates a repeatable path from experiment to service.

Simple definition

MLOps is the discipline of building and operating ML systems so that they remain useful after the first demo.

Software lifecycle versus ML lifecycle



- The ML lifecycle adds permanent dependence on data quality and drift.

Technology map: analysis and experimentation

Python	General-purpose language used across the whole stack.
Jupyter	Interactive environment for exploration, narrative explanation, and teaching labs.
pandas	Tabular data manipulation, cleaning, joins, missing-value analysis, and time handling.
Matplotlib / seaborn	Static plots for EDA, diagnostics, and teaching.
scikit-learn	Classical ML toolkit for preprocessing, pipelines, model training, evaluation, and persistence.

Repo activity

These appear directly in the notebooks folder and in the fraud modeling workflow.

Technology map: serving and interaction

Flask	Lightweight Python web framework used here to expose prediction and streaming APIs.
Streamlit	Rapid framework for interactive data apps and teaching dashboards.
River	Online machine learning library for incremental learning and drift-aware workflows.
Open-Meteo API	External weather data source used in the streaming demo.
JSON / HTTP	The transport layer connecting model services to clients and dashboards.

How they connect

Flask exposes model or sensor endpoints; Streamlit consumes them for interfaces; River updates models one observation at a time in streaming settings.

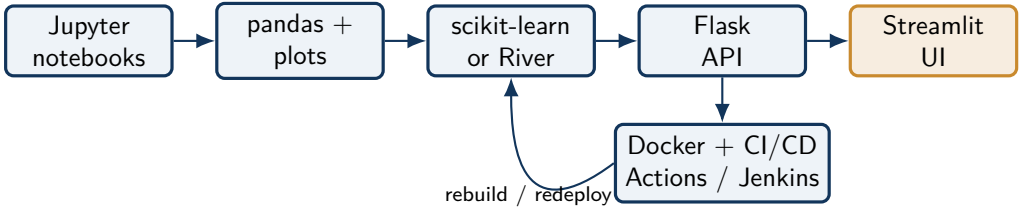
Technology map: packaging and automation

Docker	Packages applications and dependencies into reproducible containers.
GitHub Actions	Repository-native automation for tests, builds, and simple CI/CD.
Jenkins	Flexible automation server for more customizable pipelines and deployment workflows.
Git	Version control for code, and the base coordination layer for team work.

Why students should care

Without packaging and automation, repeated delivery becomes manual, slow, and error-prone.

End-to-end view of the course stack



- This is the conceptual backbone for almost every class that follows.

Where each technology appears in the repository

- ‘notebooks/‘: Jupyter, pandas, scikit-learn, River, and experimentation material.
- ‘Class6_model_api.py‘ and ‘pasteurization/serving.py‘: Flask for prediction and streaming APIs.
- ‘streamlit/‘: interactive frontends for fraud detection and online learning demos.
- ‘tutorials/Docker‘, ‘tutorials/GitHubActions‘, ‘tutorials/Jenkins‘: reproducibility and automation material.

When to use which technology

Jupyter	When we are exploring, teaching, comparing ideas, or documenting analysis.
scikit-learn	When we need robust tabular ML pipelines and standard offline evaluation.
River	When data arrives continuously and the model must update incrementally.
Flask	When we need a programmable service endpoint that other systems can call.
Streamlit	When we want an interactive dashboard or demo quickly, with low front-end overhead.
Docker / CI tools	When we want the system to be repeatable across machines and releases.

Agile AI in practice

- Write user stories around decisions, not around algorithms.
- Deliver thin vertical slices: data ingest, baseline model, API, dashboard, monitor.
- Prefer fast feedback over premature architectural perfection.
- Keep scope visible: assumptions, risks, unknown labels, and data quality issues.
- Treat the first usable model as a learning tool, not as the final product.

Questions students should be able to answer after Class 1

- Why is a trained model only one part of a usable system?
- When would we use Flask rather than Streamlit?
- Why might River be preferred over scikit-learn in a streaming scenario?
- What problem does Docker solve that Python alone does not?
- Why do CI/CD tools matter even in an ML course?

Official technology links

Papers and materials

Jupyter install guide; pandas documentation; Matplotlib documentation; scikit-learn documentation; Flask documentation; Streamlit documentation; River documentation; Open-Meteo API docs; Docker get started; GitHub Actions docs; Jenkins documentation.

From This Class to the Next

What stays with us

- Data-driven systems must be designed across data, model, interface, delivery, and monitoring concerns.
- Each technology in the stack solves a different engineering problem, not the same one with different names.
- The running cases will let us revisit the same lifecycle from multiple technical angles.

Project use

Use this map to explain where your own project sits today and which lifecycle pieces are still missing.

Next connection

Class 2 opens the Python block itself: what Python is, why it is the working language of this course, and a first tour of the toolchain (Jupyter, pandas, scikit-learn, Flask, Streamlit) we will use for the next eleven sessions.

Course thread: The semester starts with systems thinking so that the programming instrumentation taught

Foundational readings

Papers and materials

Google Cloud: MLOps continuous delivery and automation; Continuous Delivery for Machine Learning; Hidden Technical Debt in Machine Learning Systems; Designing Machine Learning Systems; Practical MLOps.